BITÁCORA DE EVIDENCIAS

# Taller 4 · Memoria del proyecto, skills y modo plan

Paso a paso del trabajo en el proyecto FinanzIA con Claude Code, contado con las capturas reales tomadas durante la sesión.

---


| | |
|---|---|
| **Grupo** | 6 |
| **Integrantes** | Angelo David Díaz Cortés · André Ricardo Rodríguez Fernández · Elian Danilo Romero Herrera |
| **Programa** | Diplomado en Ingeniería de Software con IA y Sistemas Agénticos · FET |
| **Herramienta** | Claude Code v2.1.318 (Opus 5.5, Claude Pro) en la terminal de VS Code · Windows / PowerShell |
| **Repositorio** | https://github.com/angelodiaz2001/finanzIA |
| **Fecha** | 4 de noviembre de 2026 |


**Contenido**


Fase 0 · Preparación del entorno (Python, dependencias y Claude Code)

Fase 1 · Memoria del proyecto y skills (rama, commit y PR #1)

Fase 2 · Verificación de la memoria y de las skills

Fase 3 · Solicitud 1: endpoint de comercios en modo plan (PR #2)

Fase 4 · Solicitud 2: tarjeta del Panel en modo plan (PR #3)


*Las figuras son capturas de pantalla tomadas en el equipo de trabajo. Los recuadros oscuros reproducen texto copiado tal cual de la terminal de PowerShell.*

# Fase 0 · Preparación del entorno

El proyecto se clonó en C:\Users\USUARIO\finanzIA. Faltaban Python y Claude Code, así que los instalamos y preparamos el entorno virtual antes de empezar el taller.

## 0.1 Primer intento de correr las pruebas: no existía el entorno virtual

*Salida de PowerShell*

```
PS C:\Users\USUARIO\finanzIA> .venv\Scripts\python -m pytest
.venv\Scripts\python : El módulo '.venv' no pudo cargarse.
PS C:\Users\USUARIO\finanzIA> py -0
py : El término 'py' no se reconoce como nombre de un cmdlet...
PS C:\Users\USUARIO\finanzIA> node --version
v22.14.0
PS C:\Users\USUARIO\finanzIA> git --version
git version 2.48.1.windows.1
```

Diagnóstico: Node y Git estaban instalados, pero faltaban Python y Claude Code.

## 0.2 Instalación de Python 3.13 y creación del entorno virtual

*Salida de PowerShell*

```
PS C:\Users\USUARIO\finanzIA> winget install -e --id Python.Python.3.13
Encontrado Python 3.13 [Python.Python.3.13] Versión 3.13.17
El hash del instalador se verificó correctamente
Instalado correctamente
PS C:\Users\USUARIO\finanzIA> & "$env:LOCALAPPDATA\Programs\Python\Python313\python.exe" -m venv .venv
PS C:\Users\USUARIO\finanzIA> .\.venv\Scripts\python.exe --version
Python 3.13.17
PS C:\Users\USUARIO\finanzIA> .\.venv\Scripts\python.exe -m pip install --upgrade pip
Successfully installed pip-26.3
```

## 0.3 Dependencias del backend y del frontend, y pruebas en verde

*Salida de PowerShell*

```
PS C:\Users\USUARIO\finanzIA> .\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt -r
services\requirements.txt
PS C:\Users\USUARIO\finanzIA> cd frontend; npm install; cd ..
PS C:\Users\USUARIO\finanzIA> .\.venv\Scripts\python.exe -m pytest
........................                                                 [100%]
24 passed in 0.71s
```

Punto de partida: 24 pruebas existentes en verde.

## 0.4 Instalación e inicio de Claude Code

Instalamos Claude Code con el instalador oficial para PowerShell. Al abrirlo en la carpeta del proyecto, confirmamos que es un proyecto de confianza.




**Figura 1.** *Claude Code pide confirmar que la carpeta del proyecto es de confianza.*

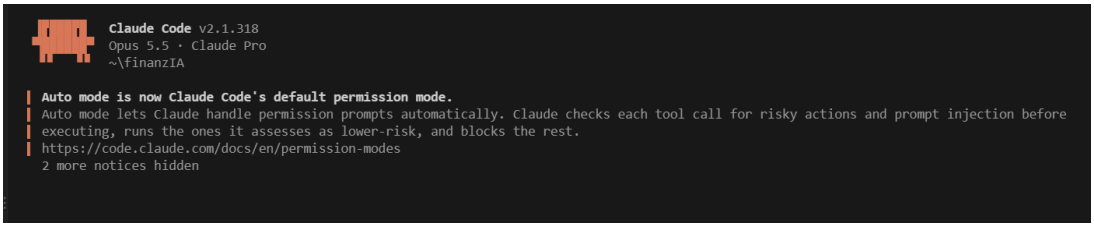



**Figura 2.** *Claude Code iniciado en ~\finanzIA con Opus 5.5 y la cuenta Claude Pro.*

# Fase 1 · Memoria del proyecto y skills

Creamos la rama chore/memoria-y-skills, copiamos los tres CLAUDE.md y las dos skills, los revisamos y los llevamos a main por pull request.

## 1.1 Rama de trabajo y archivos copiados

*Salida de PowerShell*

```
PS C:\Users\USUARIO\finanzIA> git switch -c chore/memoria-y-skills
PS C:\Users\USUARIO\finanzIA> git status
On branch chore/memoria-y-skills
Untracked files:
        CLAUDE.md
        backend/.claude/
        backend/CLAUDE.md
        frontend/.claude/
        frontend/CLAUDE.md
```

npm install había modificado `frontend/package-lock.json`. Lo restauramos con `git restore` para que no entrara en un commit que no le correspondía.

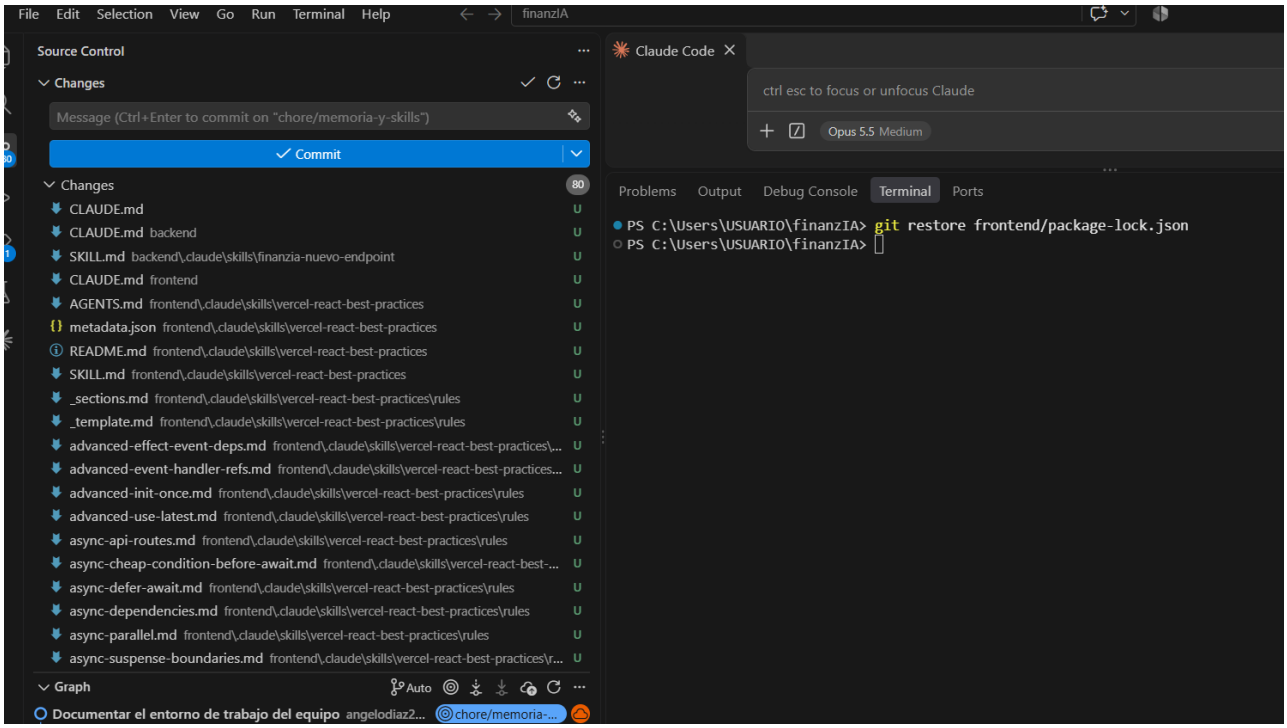



**Figura 3.** *VS Code: los archivos nuevos, 80 en total porque la skill de Vercel trae 70 reglas en Markdown, más su plantilla y su índice. Revisamos los 3 CLAUDE.md y los 2 SKILL.md.*

## 1.2 Commit y push de la rama

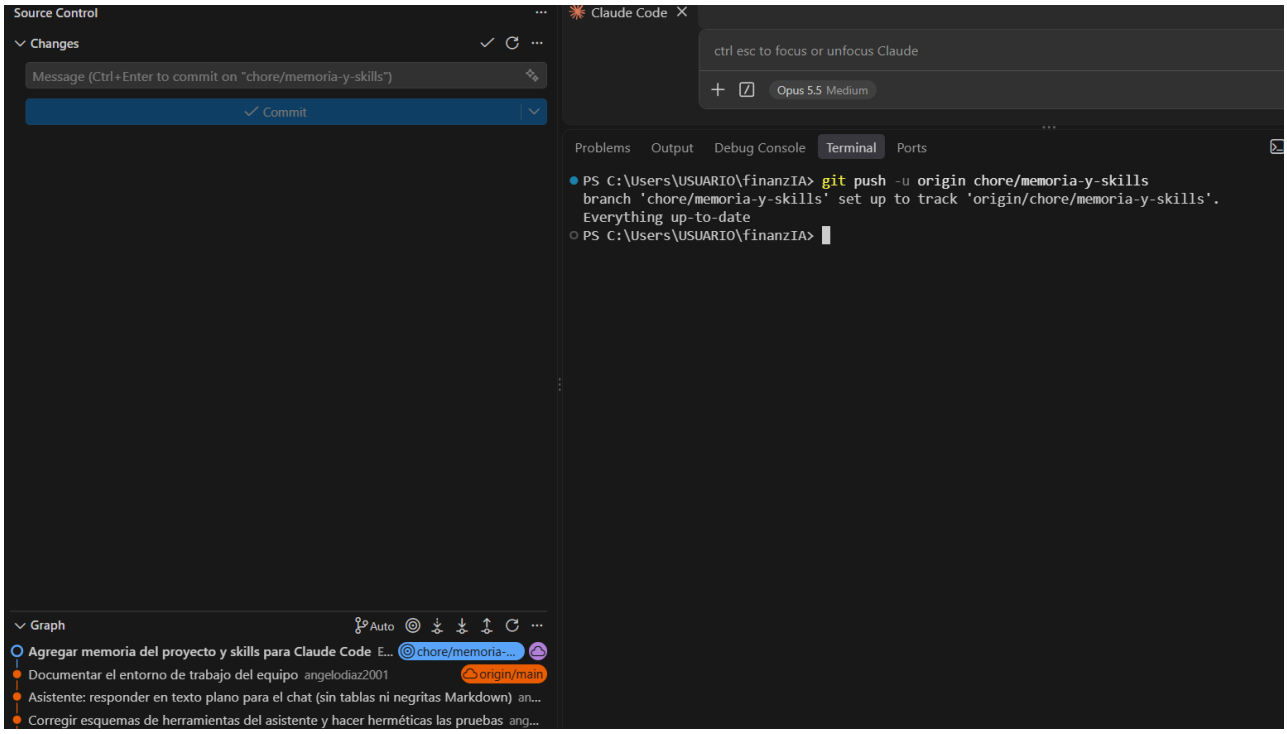



**Figura 4.** *Push de chore/memoria-y-skills; en el grafo se ve el commit "Agregar memoria del proyecto y skills para Claude Code".*

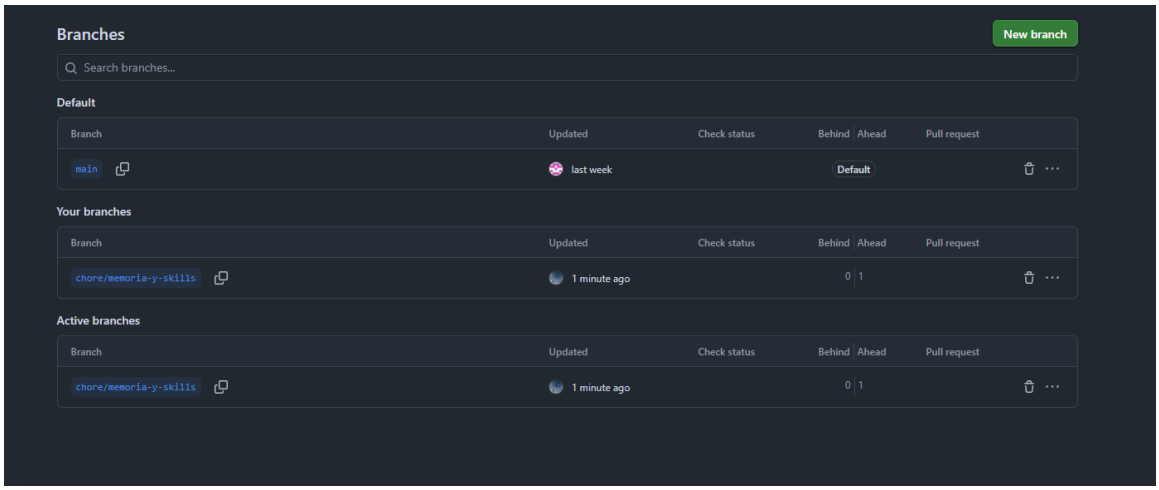



**Figura 5.** *GitHub: la rama publicada, con 1 commit por delante de main.*

## 1.3 Pull request #1 y merge

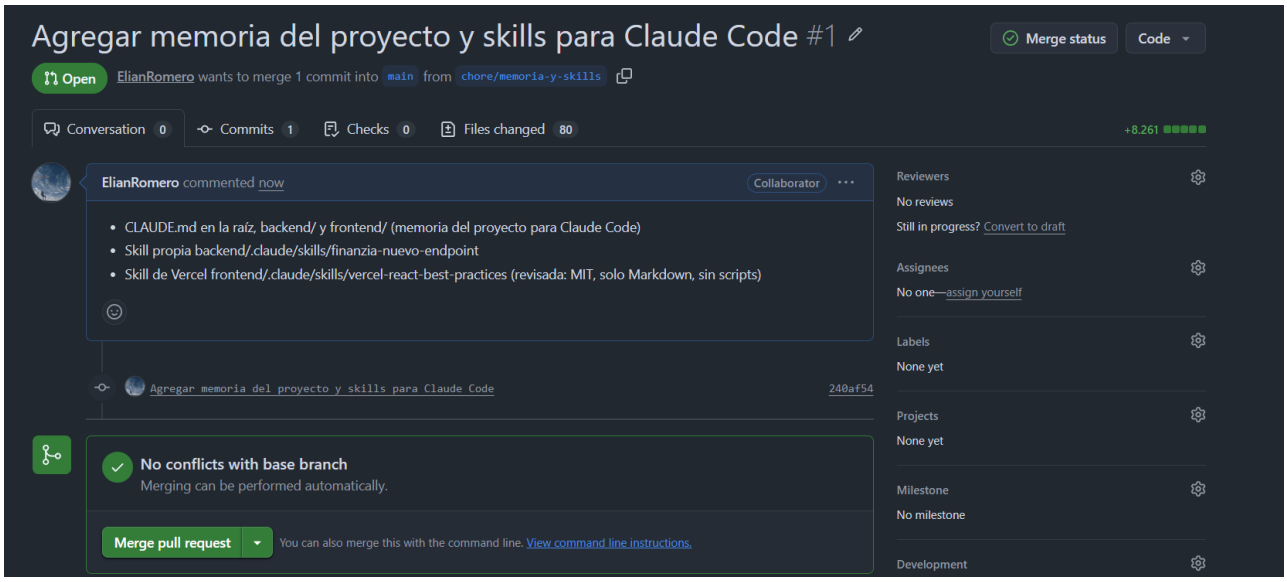



**Figura 6.** *PR #1 abierto, con la descripción del contenido y la revisión de la skill de Vercel.*

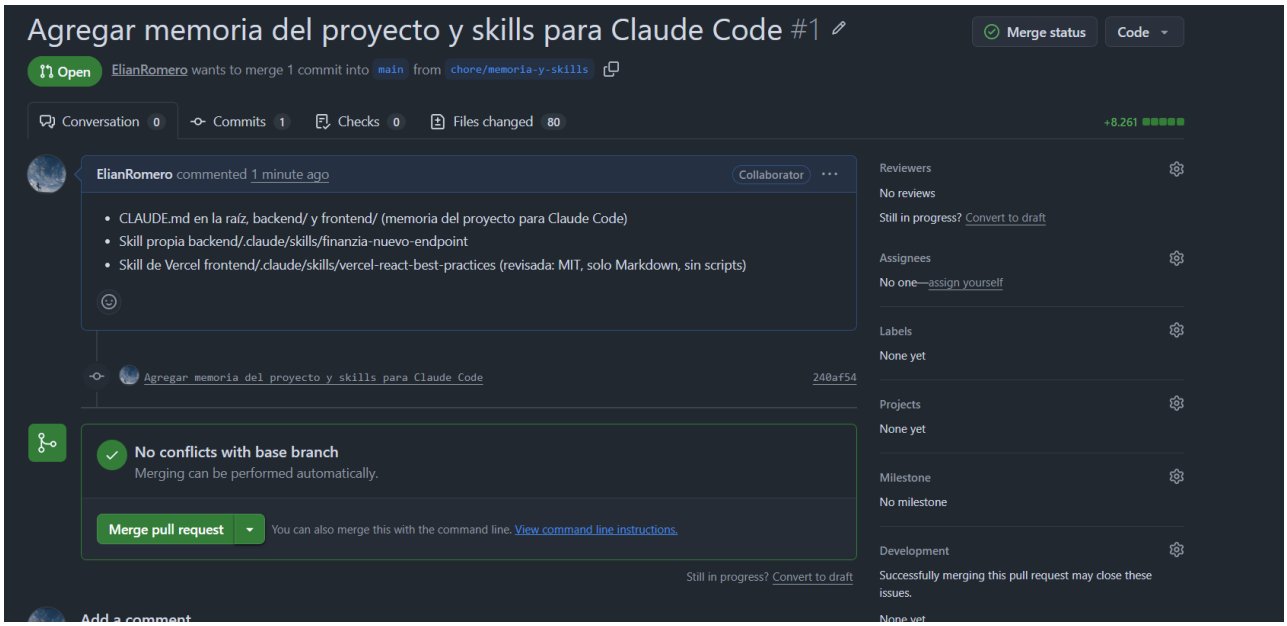



**Figura 7.** *PR #1 sin conflictos, listo para fusionar.*

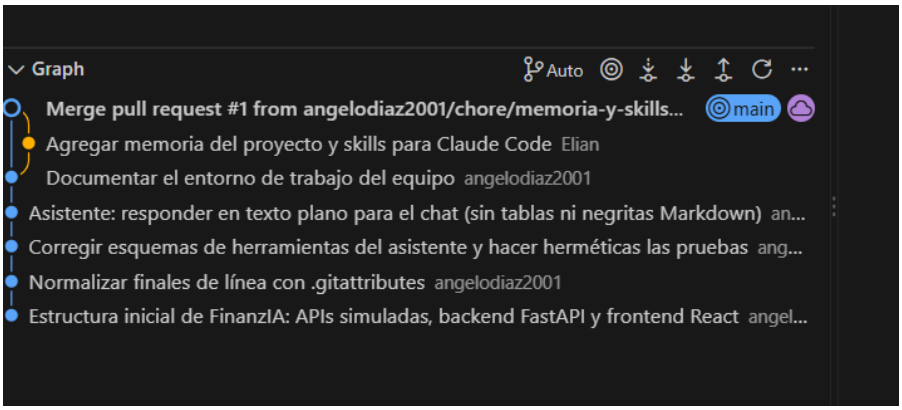



**Figura 8.** *Después del merge: main local actualizado con "Merge pull request #1".*

# Fase 2 · Verificación de la memoria y de las skills

Comprobamos que Claude Code lee la memoria del proyecto y que cada skill se carga y se usa en su parte del monorepo.

## 2.1 La memoria funciona

Le preguntamos a Claude Code cómo ejecutar las pruebas. Respondió sin buscar nada, con datos que solo están en CLAUDE.md: el comando exacto, que el .venv está en la raíz y que el frontend se verifica con npm run build.

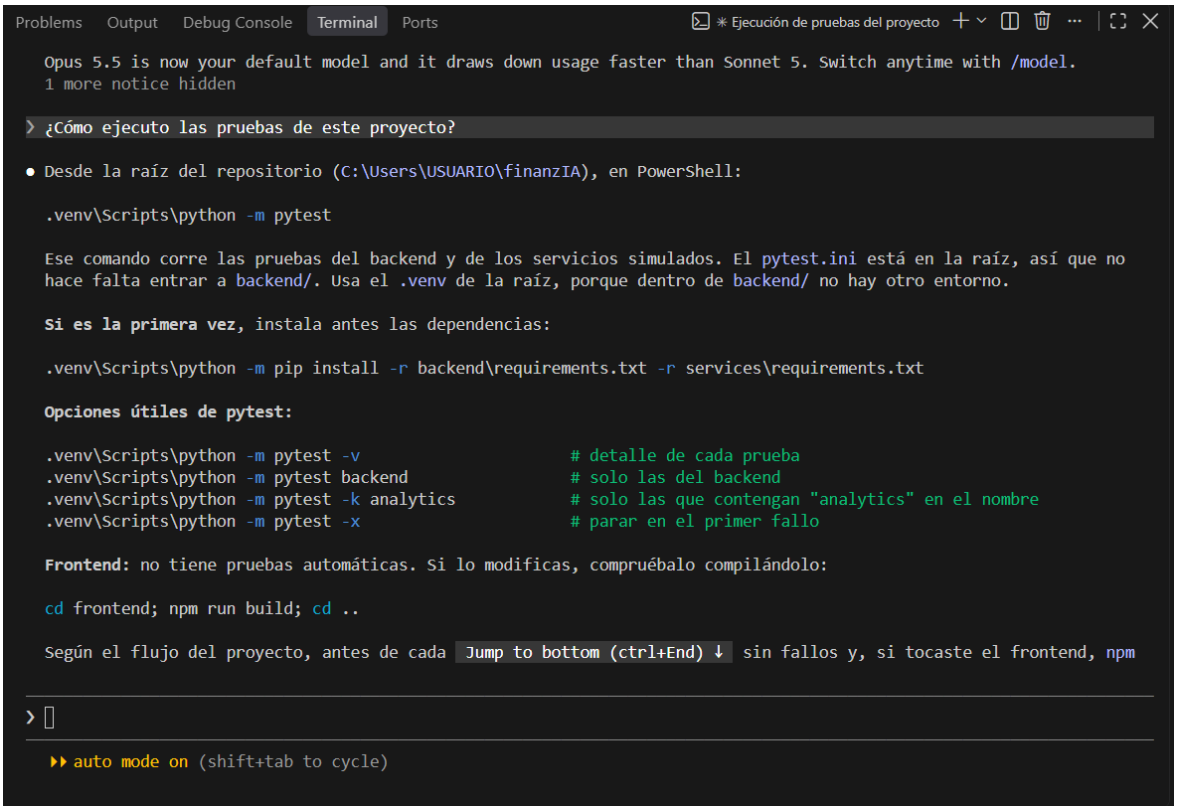



**Figura 9.** *Respuesta de Claude Code basada en la memoria del proyecto.*

## 2.2 Skill del backend: finanzia-nuevo-endpoint

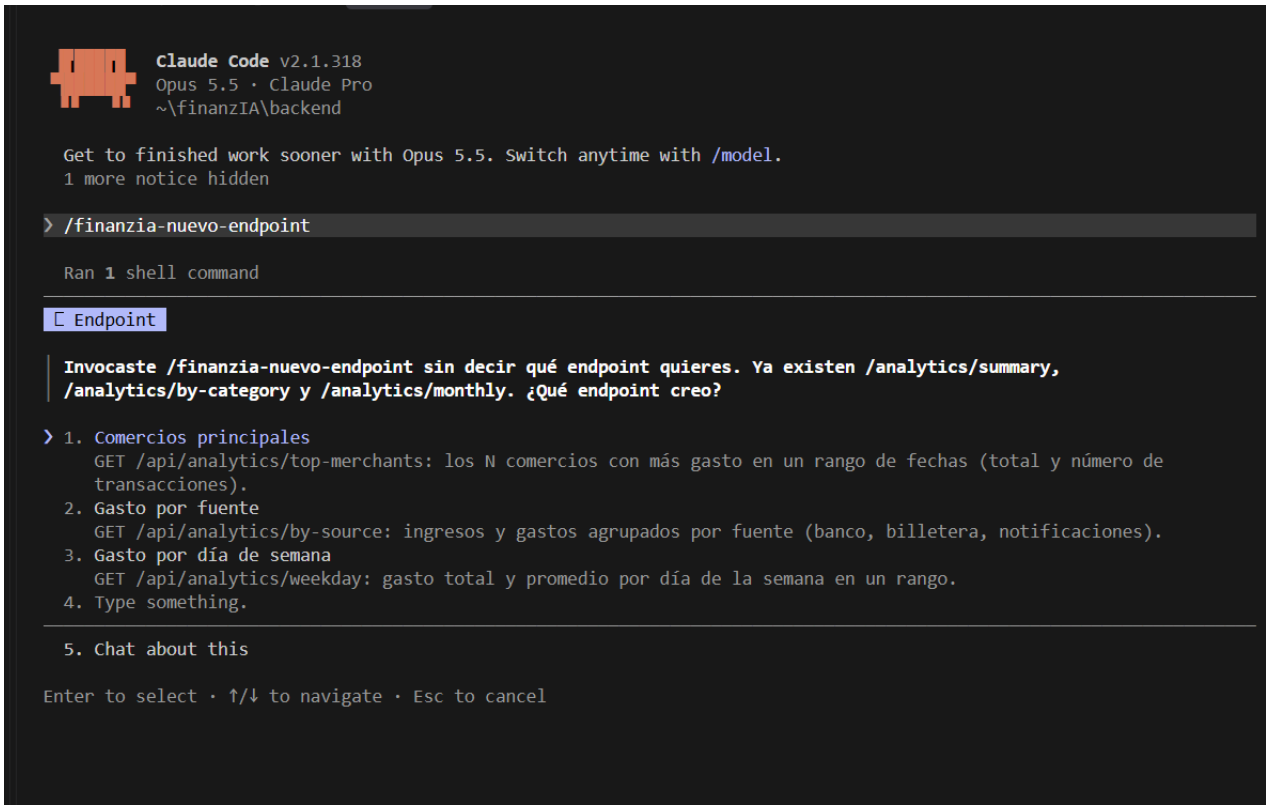



**Figura 10.** *Claude Code abierto en backend/: la skill se invoca y reconoce los endpoints que ya existen. Cancelamos las opciones porque el endpoint se hizo después, en modo plan.*

## 2.3 Skill del frontend: vercel-react-best-practices

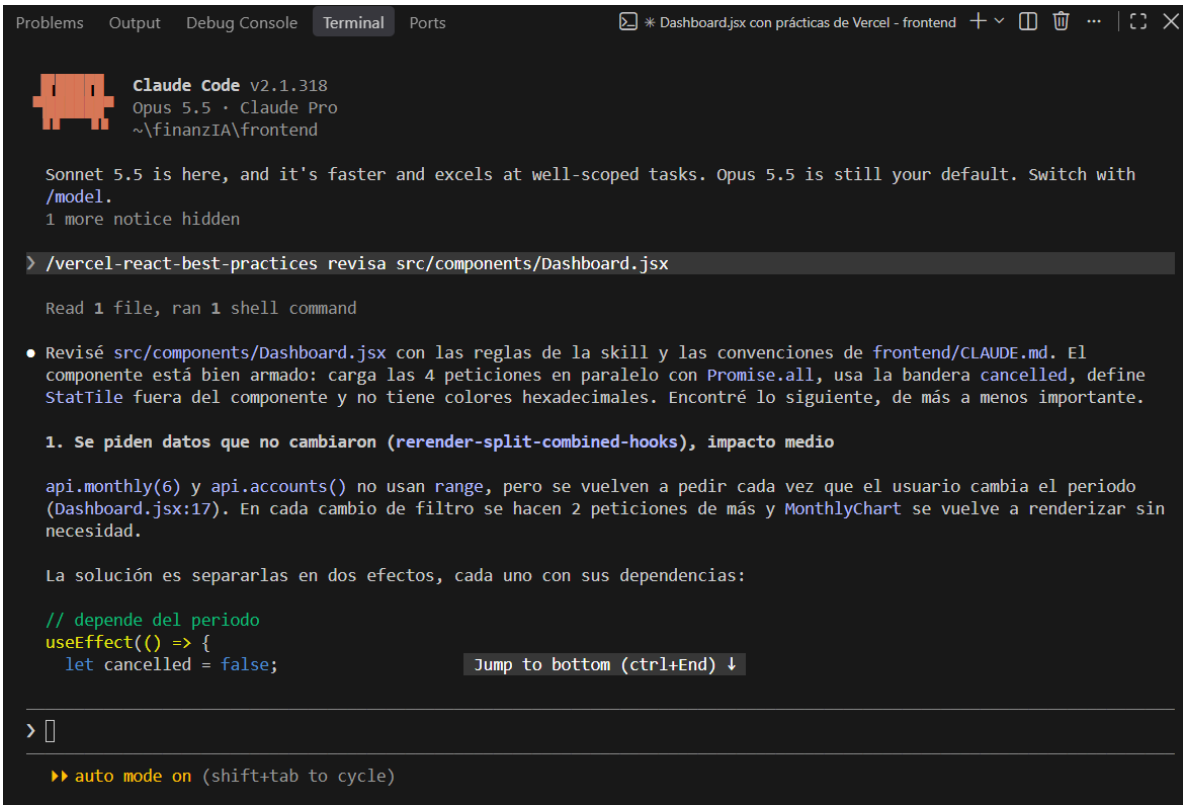



**Figura 11.** *Revisión de Dashboard.jsx con la skill de Vercel, usando sus reglas (rerender-split-combined-hooks).*

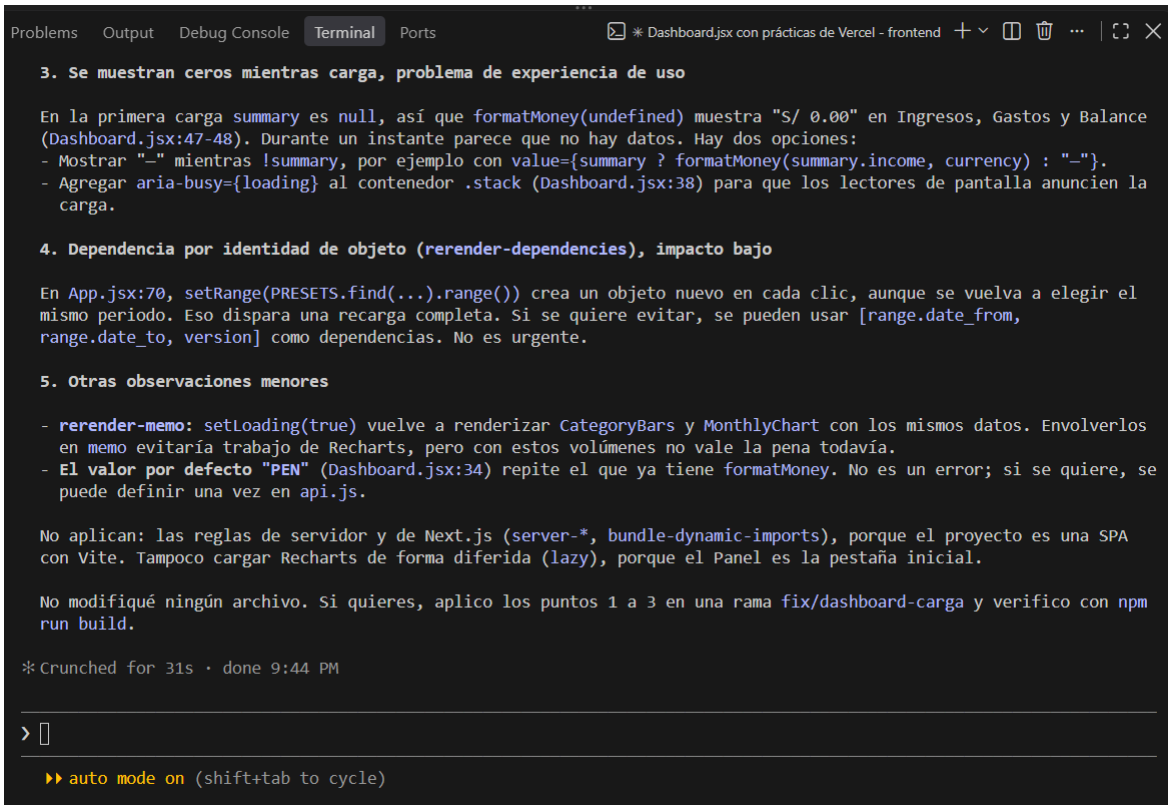



**Figura 12.** *Final de la revisión: dice qué reglas no aplican a una SPA con Vite y confirma que no modificó archivos.*

# Fase 3 · Solicitud 1: endpoint de comercios con mayor gasto

Historia: como usuario de FinanzIA, quiero saber en qué comercios gasto más dinero en un periodo, para identificar dónde puedo recortar gastos.

## 3.1 Rama y prompt en modo plan

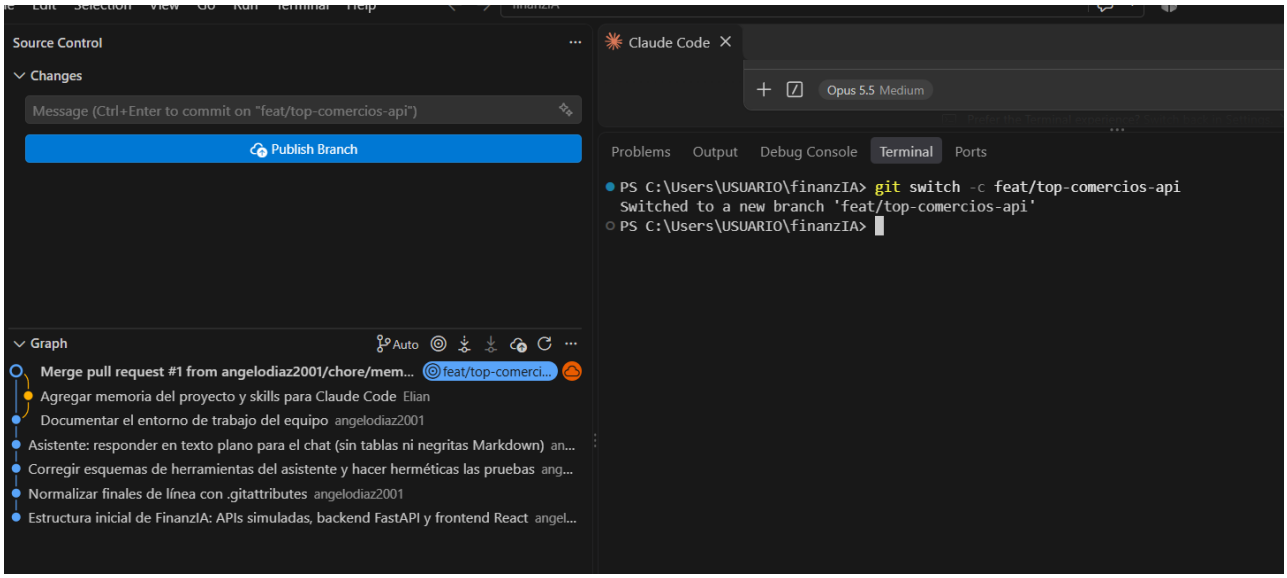



**Figura 13.** *Rama feat/top-comercios-api creada desde main.*

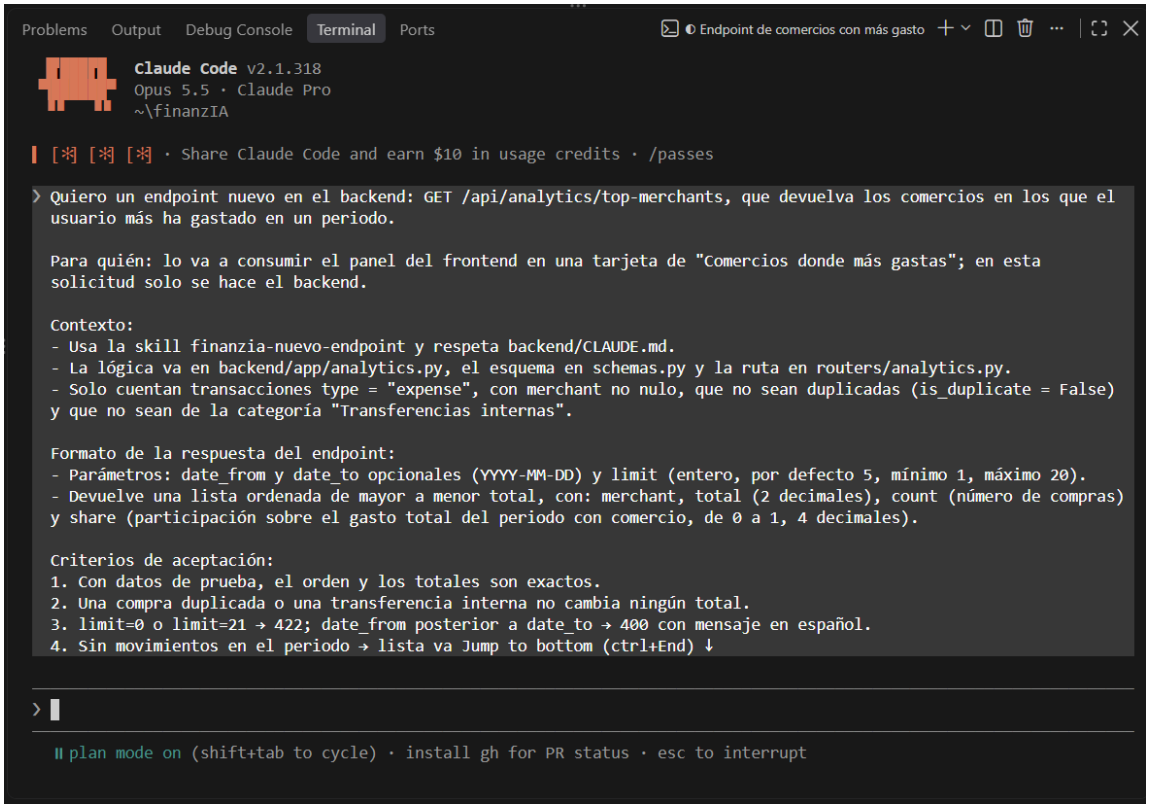



**Figura 14.** *Prompt pegado con "plan mode on".*

## 3.2 Control de permisos: el agente quiso buscar fuera del repositorio

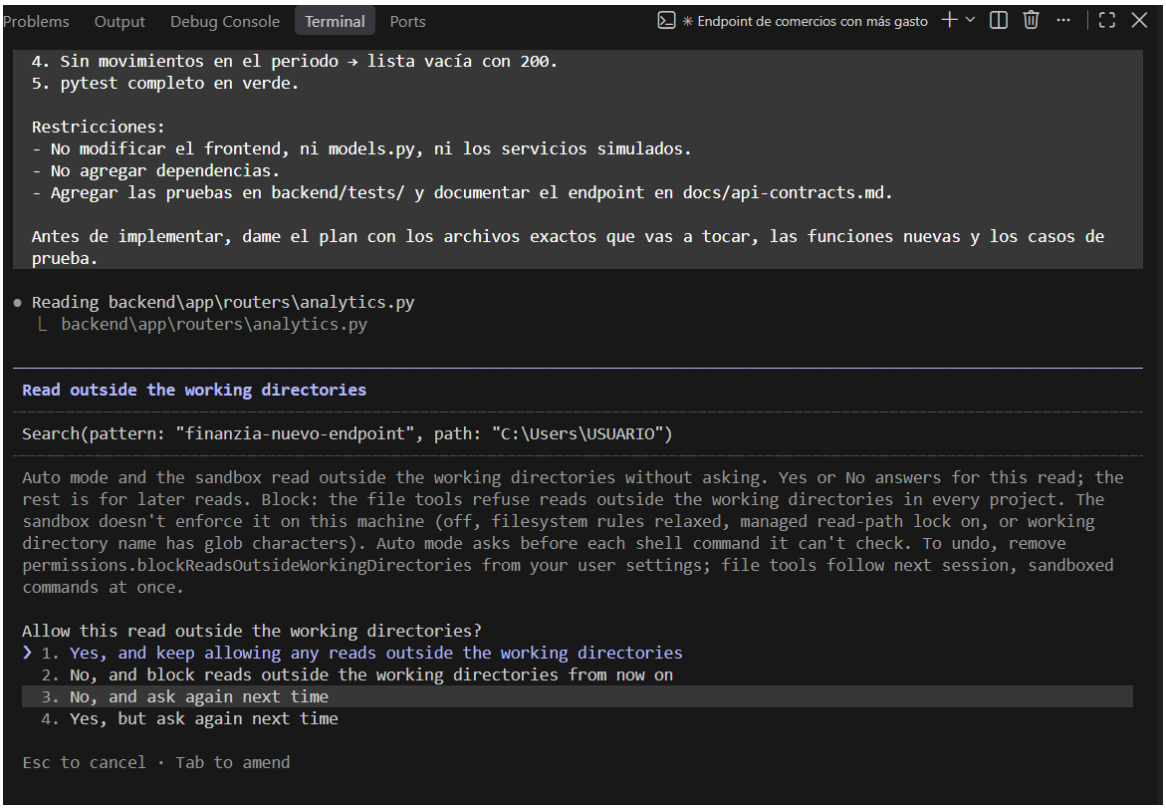



**Figura 15.** *Claude intentó buscar la skill en toda la carpeta del usuario. Le negamos el acceso (opción 3).*

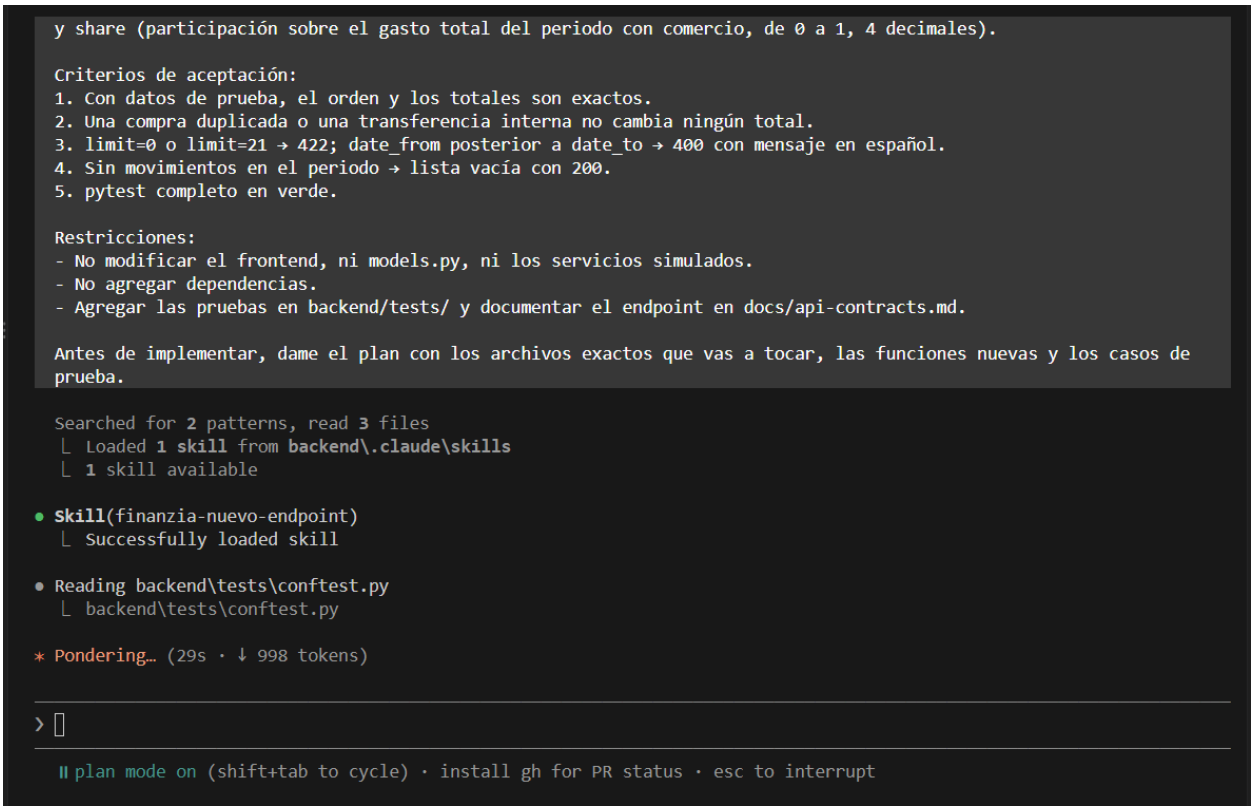



**Figura 16.** *El agente encontró la skill dentro del proyecto: "Skill(finanzia-nuevo-endpoint) · Successfully loaded skill".*

### 3.3 Revisión del plan y corrección pedida al agente

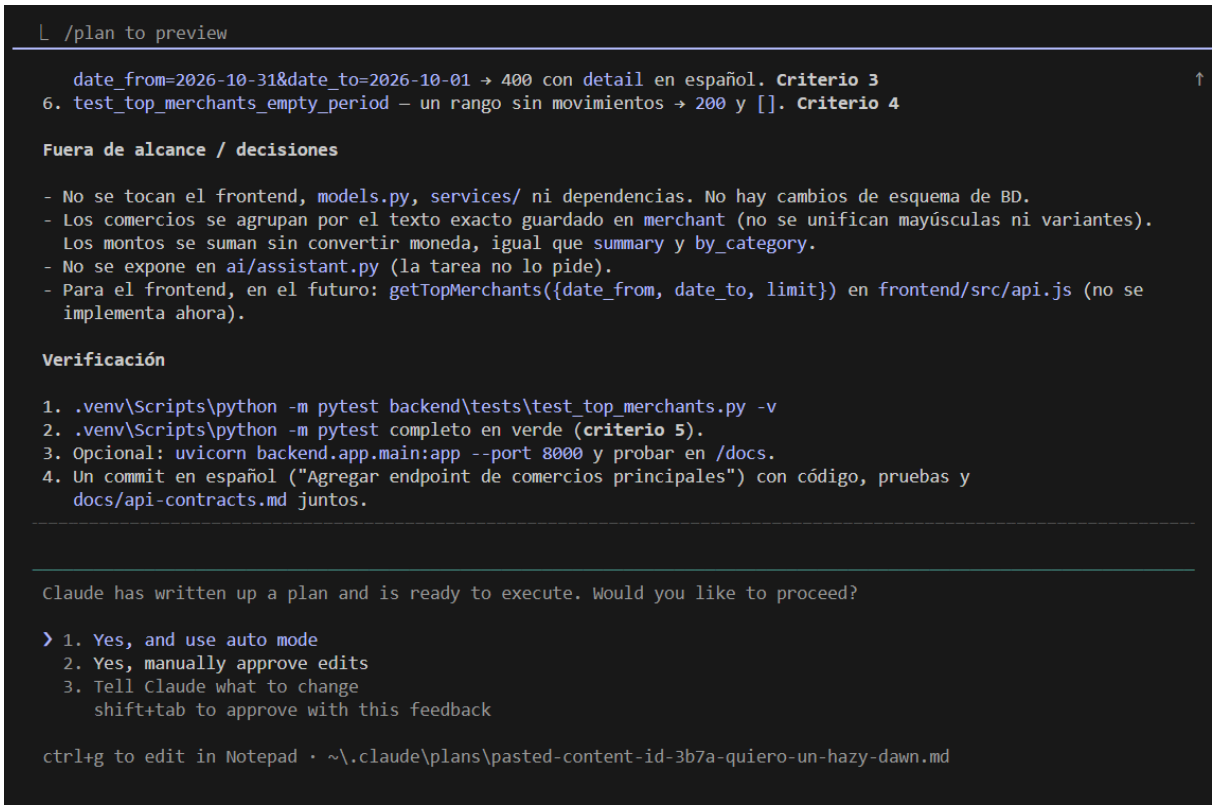



**Figura 17.** *Primer plan: 6 pruebas enlazadas a los criterios y verificación con pytest.*

**Error detectado:** en la billetera, los envíos de dinero a personas ("Envío a Lucía") se guardan con merchant = nombre de la persona. Con el primer plan esas personas habrían salido como "comercios". No editamos el plan a mano: se lo pedimos al agente.

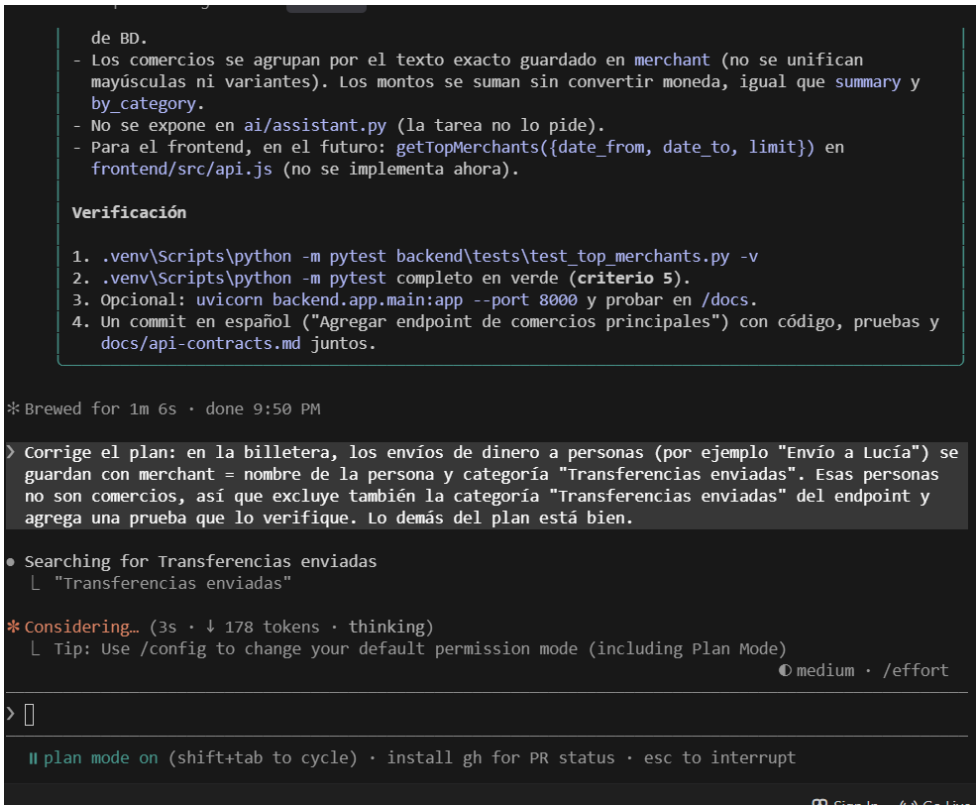



**Figura 18.** *Mensaje de corrección: excluir la categoría "Transferencias enviadas" y agregar una prueba.*

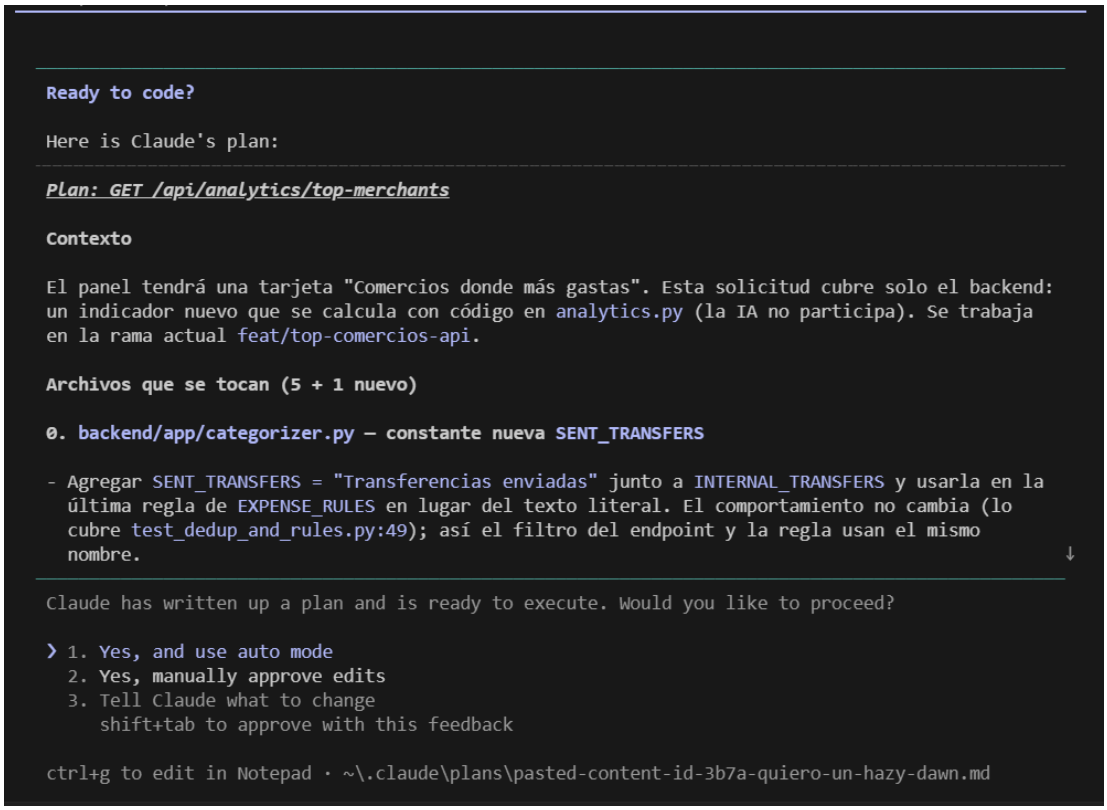



**Figura 19.** *Plan corregido: agrega la constante SENT_TRANSFERS. Lo copiamos sin editar antes de aprobarlo (después se renombró a plan-1-grupo6.md).*

## 3.4 Implementación

```
Problems  Output  Debug Console  Terminal  Ports

● Write(backend\tests\test_top_merchants.py)
  └ Wrote 107 lines to backend\tests\test_top_merchants.py
     1 """Pruebas del endpoint GET /api/analytics/top-merchants."""
     2 from __future__ import annotations
     3
     4 from datetime import datetime
     5 from itertools import count
     6 from typing import Optional
     7
     8 import pytest
     9
    10 from backend.app.categorizer import INTERNAL_TRANSFERS, SENT_TRANSFERS
    … +97 lines

  Ran 2 shell commands

● Las 33 pruebas pasan. Hago el commit en la rama, como decía el plan aprobado.

● Staging changes and committing
  └ $ git status --short; git add backend/app/categorizer.py backend/app/analytics.py
    backend/app/schemas.py backend/app/routers/analytics.py backend/tests/test_top_merchants.py
    docs/api-contracts.md; git commit -q -m @'
    Agregar endpoint de comercios principales
    GET /api/analytics/top-merchants devuelv…

· Considering… (1m 26s · ↓ 7.8k tokens)
  └ Tip: Use /btw to ask a quick side question without interrupting Claude's current work

──────────────────────────────────────────────────────────── add-top-merchants-endpoint ─
❯
──────────────────────────────────────────────────────────────────────────────────────
  ▸▸ auto mode on (shift+tab to cycle) · install gh for PR status · esc to interrupt
```

**Figura 20.** *Claude escribe test_top_merchants.py; pasan 33 pruebas (antes 24) y hace el commit.*

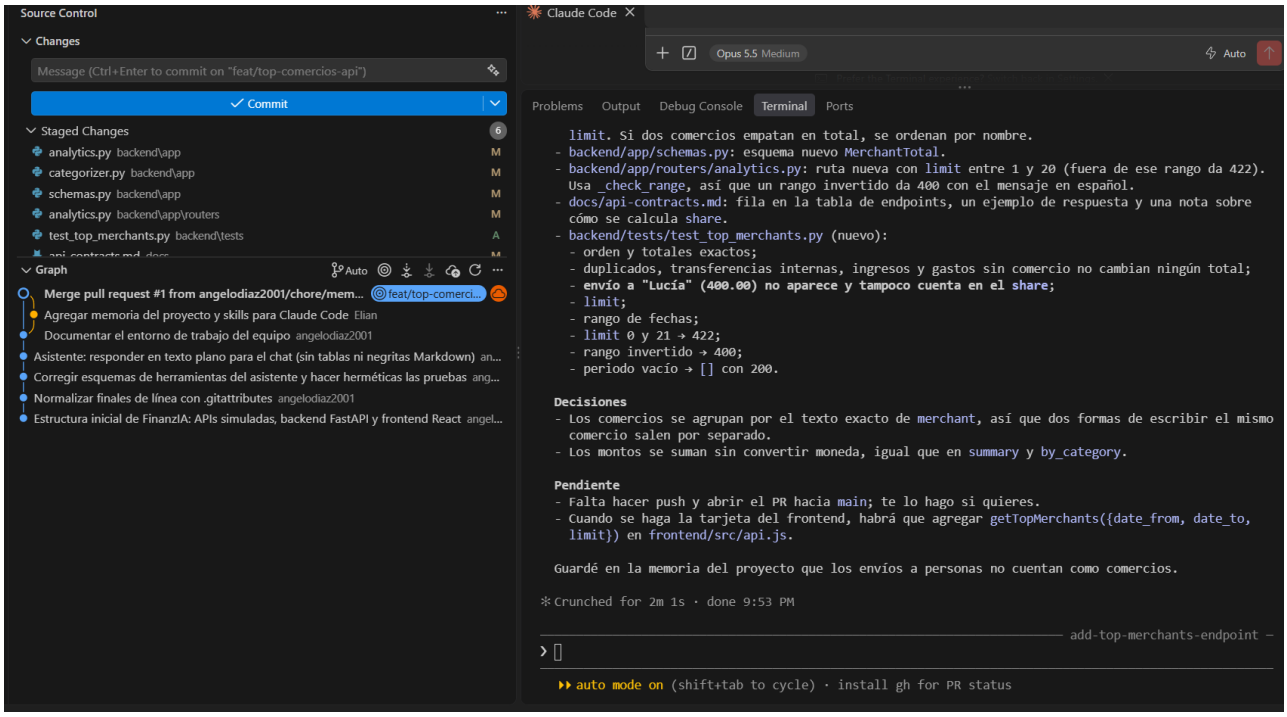



**Figura 21.** *Resumen final: 6 archivos, las pruebas por criterio y la exclusión de "Lucía".*

## 3.5 La aplicación corriendo

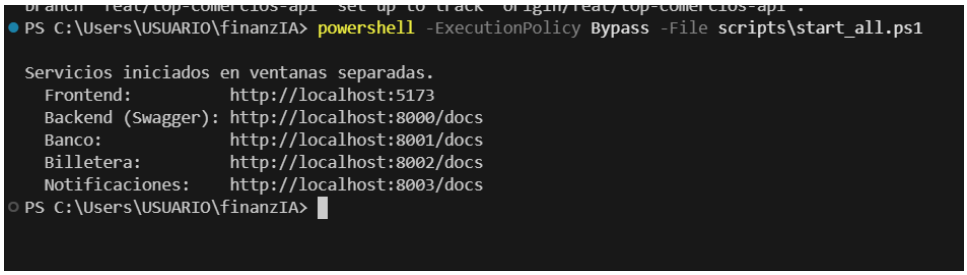



**Figura 22.** *start_all.ps1 levanta los cinco servicios.*

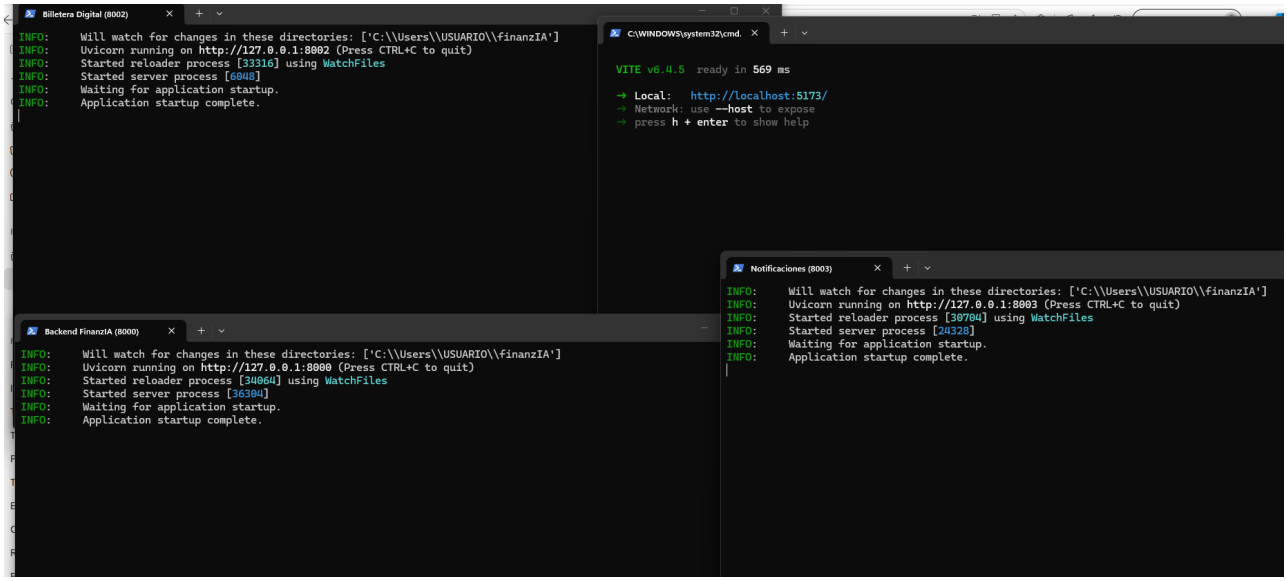



**Figura 23.** *Las ventanas de PowerShell: Billetera (8002), Notificaciones (8003), Backend (8000) y Vite (5173).*

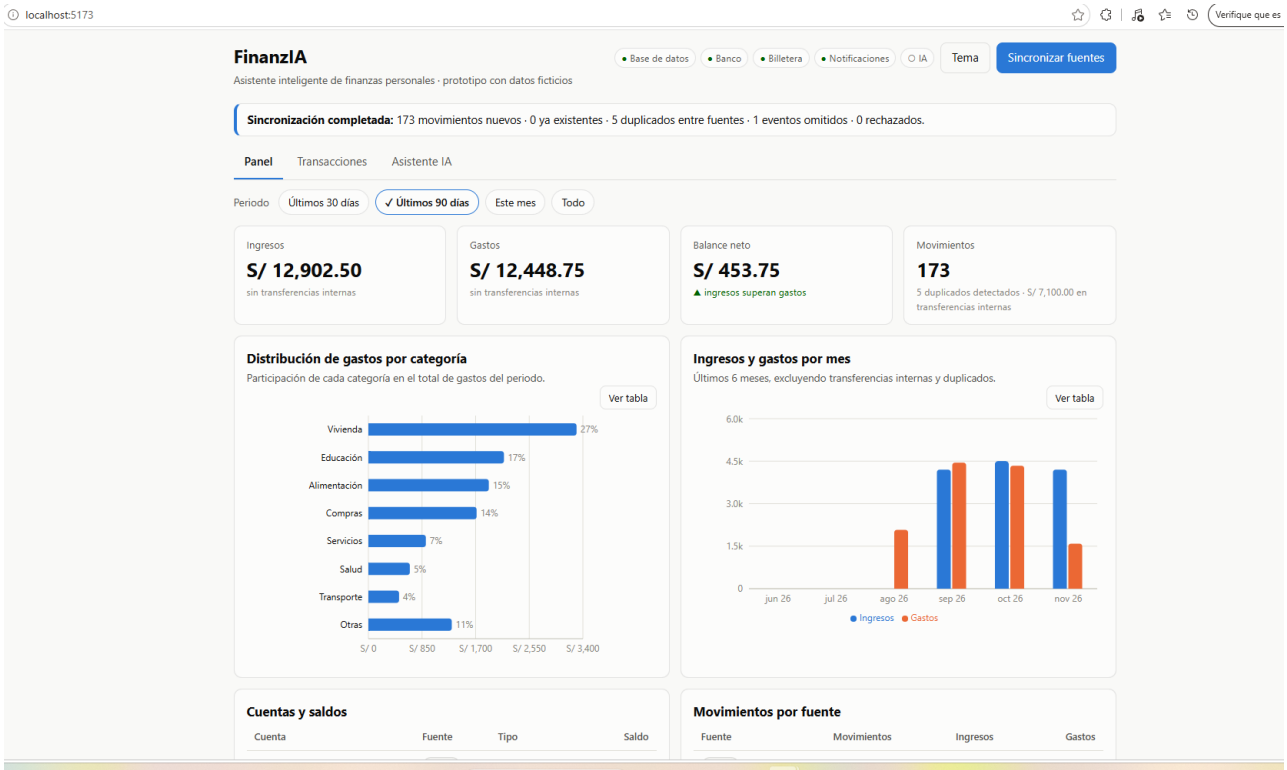



**Figura 24.** *Panel después de sincronizar: 173 movimientos nuevos y 5 duplicados entre fuentes detectados.*

## 3.6 Prueba del endpoint en Swagger

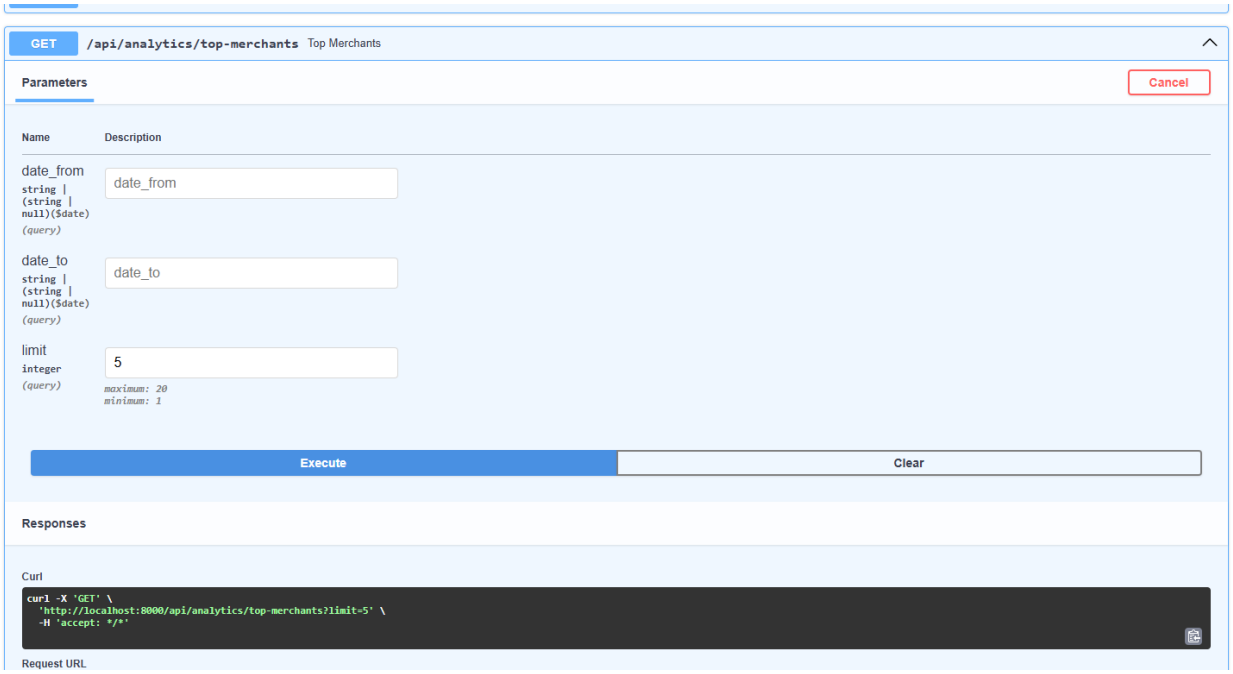



**Figura 25.** *GET /api/analytics/top-merchants en Swagger, con limit entre 1 y 20.*

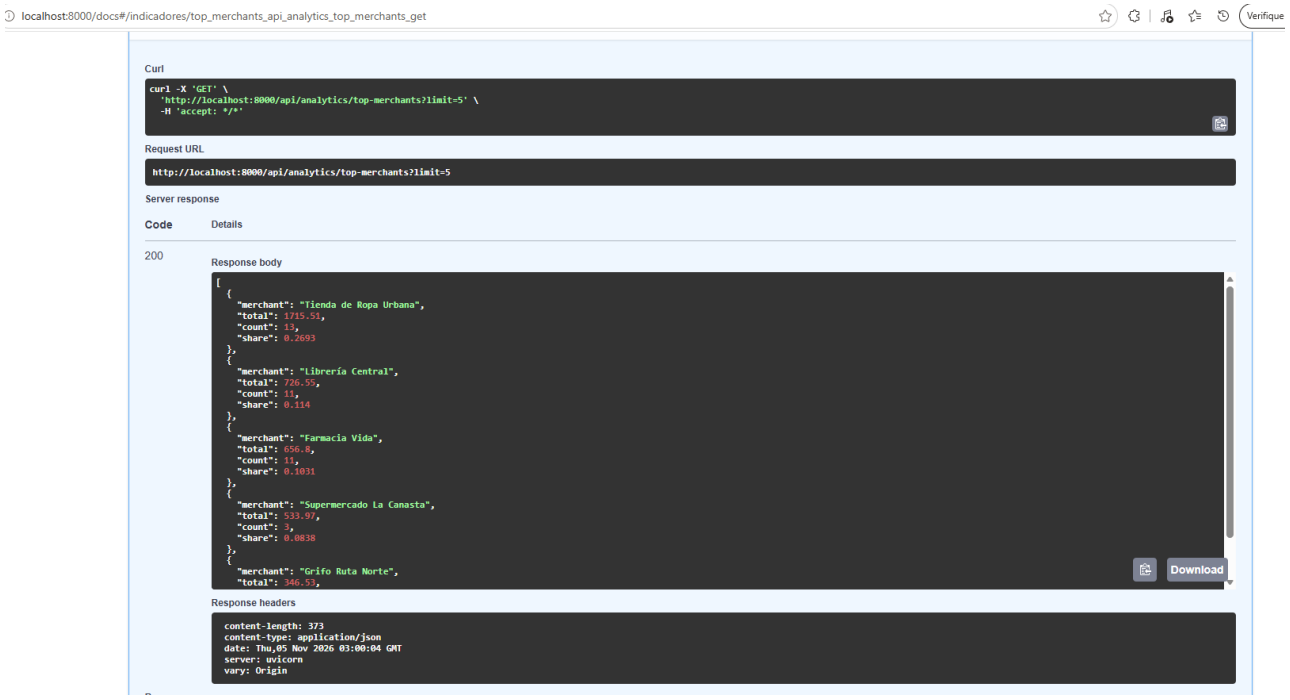



**Figura 26.** *Respuesta 200: comercios ordenados por total, con count y share, y ninguna persona.*

## 3.7 PR #2 revisado y fusionado

Antes de fusionar revisamos el diff contra el plan: coincide archivo por archivo, la lógica está en analytics.py y el router solo valida.

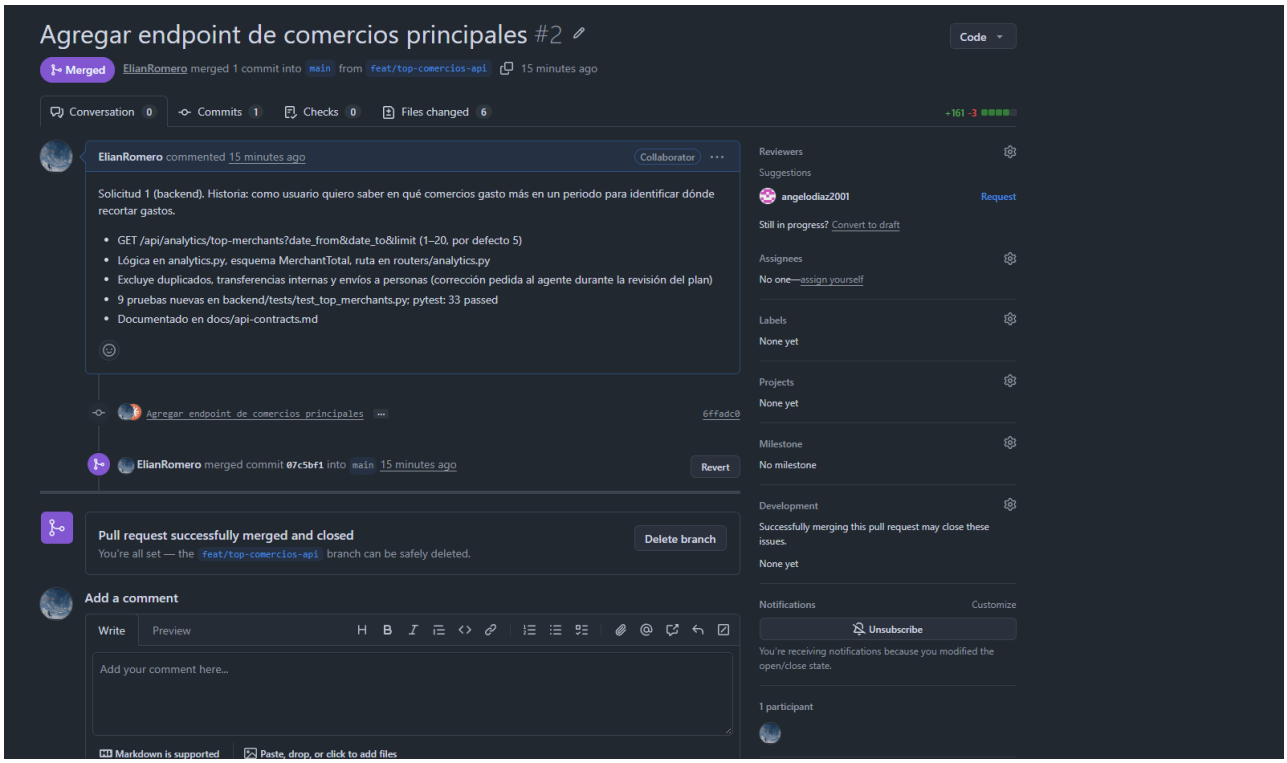



**Figura 27.** *PR #2 fusionado en main, con la historia y la corrección documentadas.*

# Fase 4 · Solicitud 2: tarjeta "Comercios donde más gastas"

Historia: como usuario de FinanzIA, quiero ver en el Panel los 5 comercios donde más gasto en el periodo seleccionado, para entender mis hábitos de consumo de un vistazo.

## 4.1 Primer intento descartado: no estaba en modo plan

En el primer intento la barra inferior decía "auto mode on". El agente escribió un plan en el chat, pero no en modo plan. Lo descartamos, confirmamos con git status que no había cambios y lo repetimos.

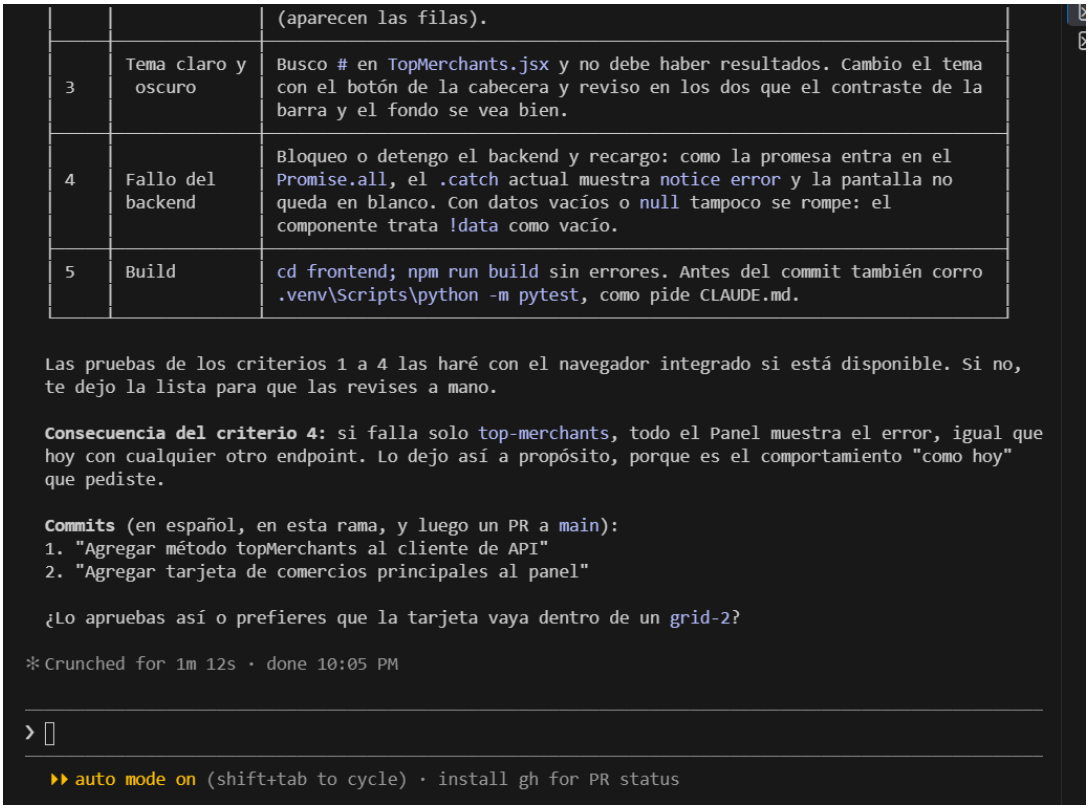



**Figura 28.** *Intento descartado: el plan sale en el chat y abajo se ve "auto mode on".*

*Salida de PowerShell*

```
PS C:\Users\USUARIO\finanzIA> git status
On branch feat/top-comercios-panel
nothing to commit, working tree clean
```

## 4.2 Modo plan activado y plan aprobado

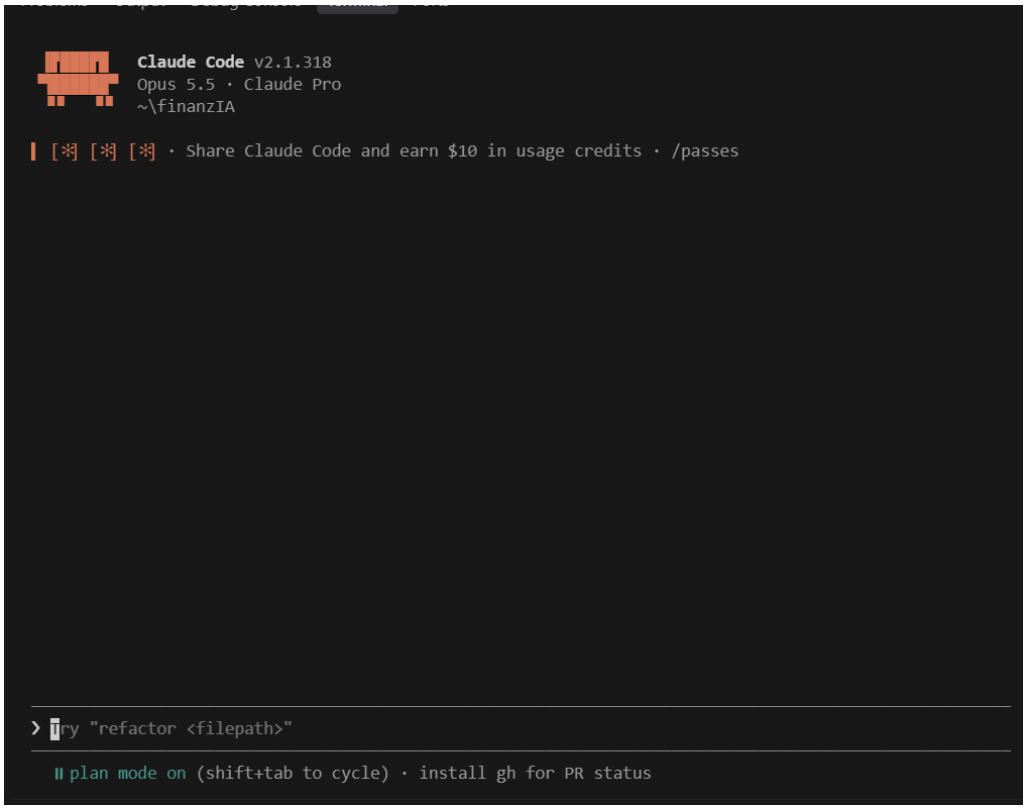



**Figura 29.** *Antes de pegar el prompt: "plan mode on".*

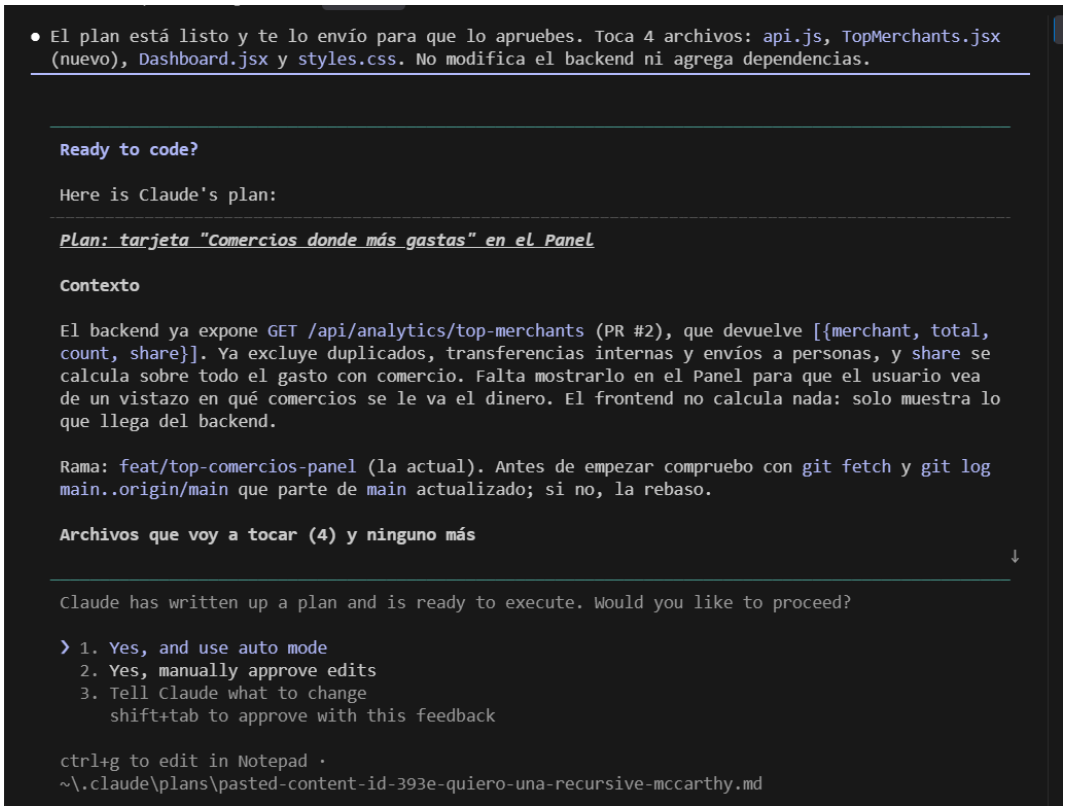



**Figura 30.** *Plan en modo plan: 4 archivos, sin backend ni dependencias nuevas. Se copió sin editar antes de aprobarlo.*

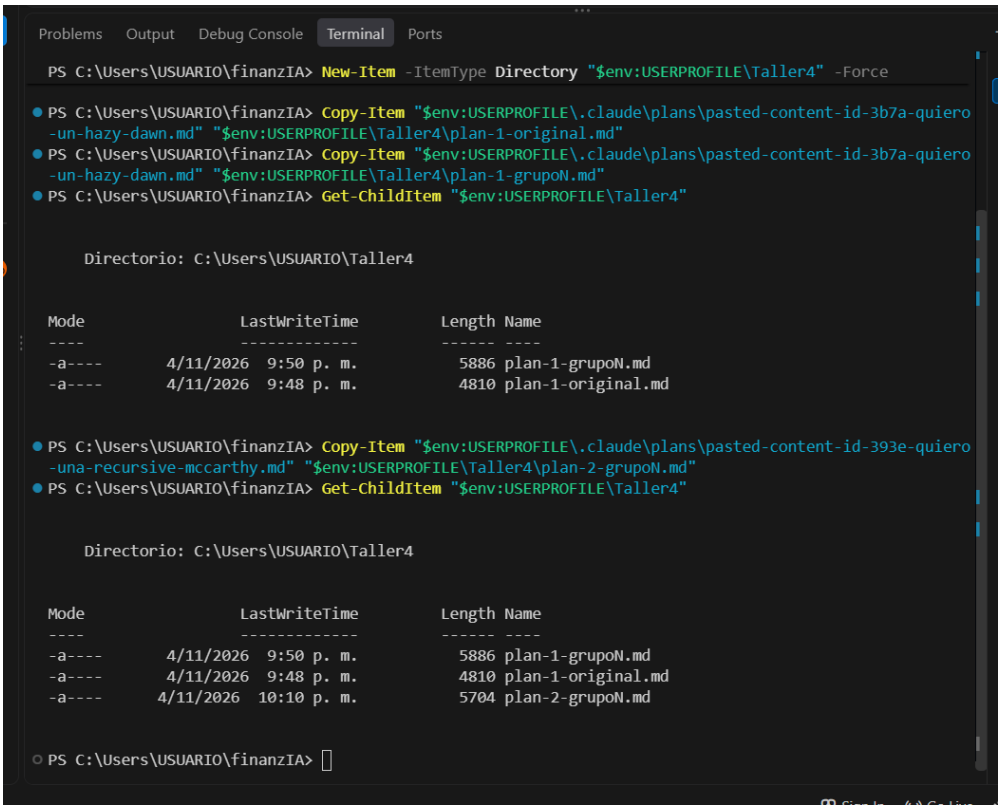



**Figura 31.** *Carpeta Taller4 con los planes copiados sin editar. Se guardaron con el nombre provisional grupoN y al final se renombraron a plan-1-grupo6.md y plan-2-grupo6.md, que son los que se entregan.*

## 4.3 Implementación y verificación

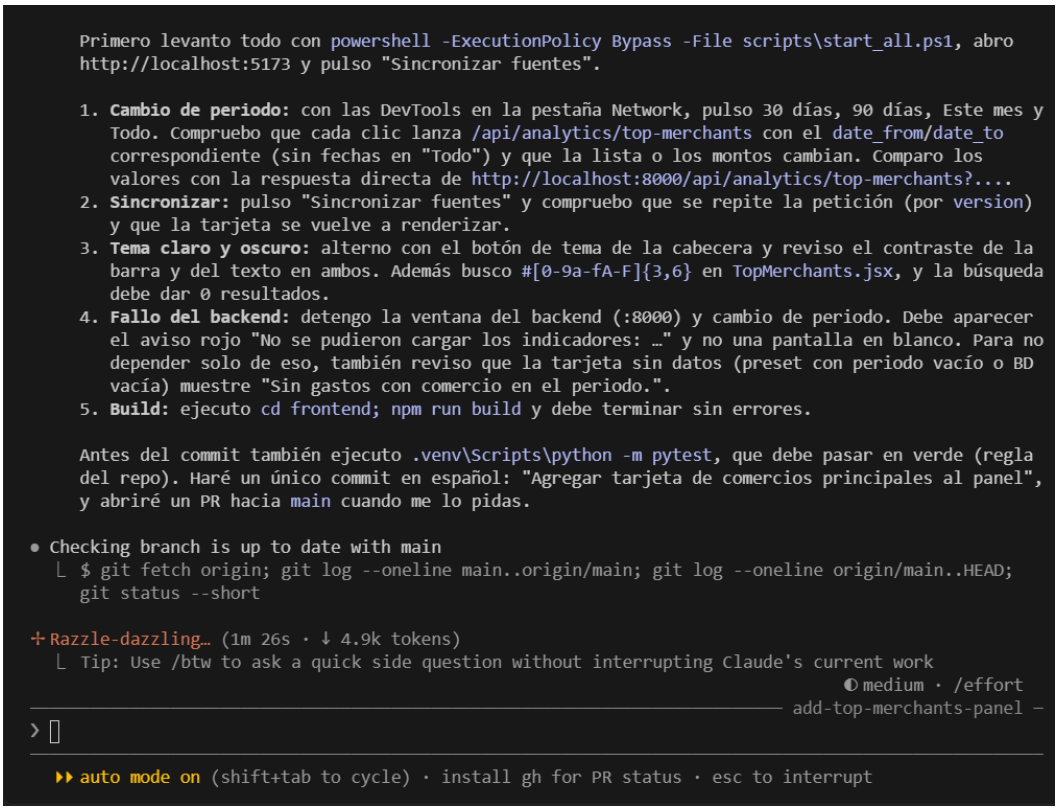



**Figura 32.** *Plan de verificación de cada criterio de aceptación.*

**Verificación por criterio:**

| Criterio | Estado |
|---|---|
| 5. npm run build | ✅ Compila sin errores. Sale un aviso de que el paquete JS pesa más de 500 kB; ya salía antes de este cambio y es casi todo Recharts. |
| 3. Sin colores hexadecimales en el componente | ✅ La búsqueda en TopMerchants.jsx da 0 resultados. |
| Pruebas del repo | ✅ Las 33 de pytest pasan. |
| 1. Cambio de periodo, 2. Sincronizar, 4. Fallo del backend, y cómo se ve en tema claro y oscuro | ⏳ Pendiente de probar en el navegador. |

Por el código, los criterios 1 y 2 deberían cumplirse porque la tarjeta usa la misma carga que el resto del Panel. Para el 4: si el backend falla, el Panel muestra el mismo aviso de error de siempre. Para probarlo levanta scripts\start_all.ps1, pulsa "Sincronizar fuentes", cambia de periodo y de tema, y luego detén el backend en el puerto 8000.

Si quieres, abro el navegador para hacer esa prueba visual, y después hago push y abro el PR hacia main.

✻ Churned for 2m 22s · done 10:12 PM

add-top-merchants-panel

›

▸▸ auto mode on (shift+tab to cycle) · install gh for PR status

**Figura 33.** *Resultado: npm run build compila, no hay colores hexadecimales y pasan las 33 pruebas.*

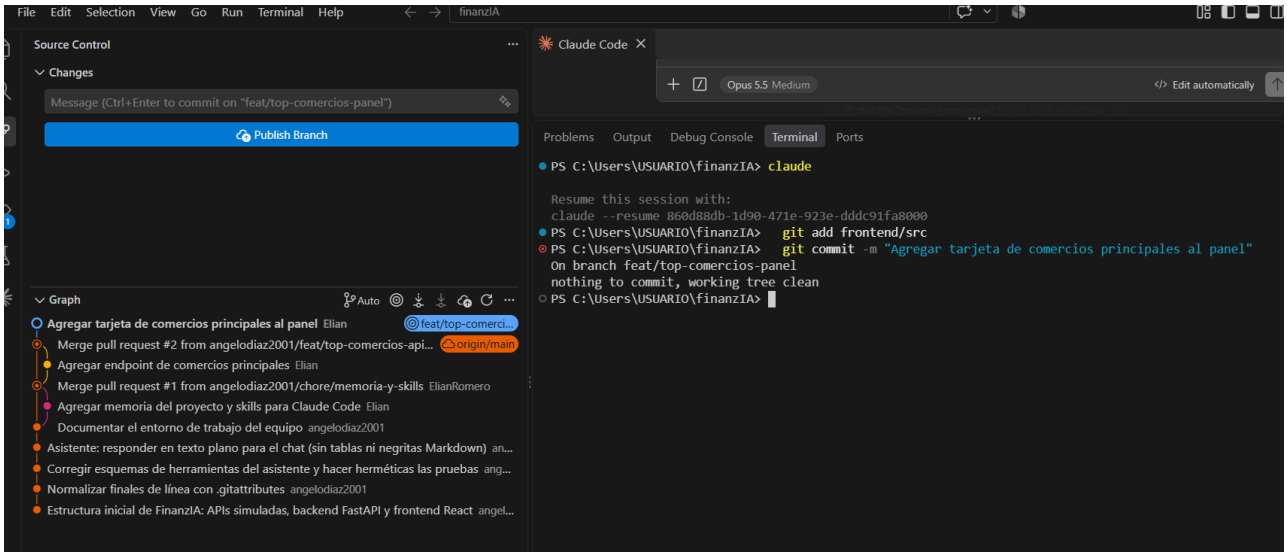



**Figura 34.** *Grafo: el commit "Agregar tarjeta de comercios principales al panel" sobre el merge del PR #2.*

## 4.4 La tarjeta funcionando

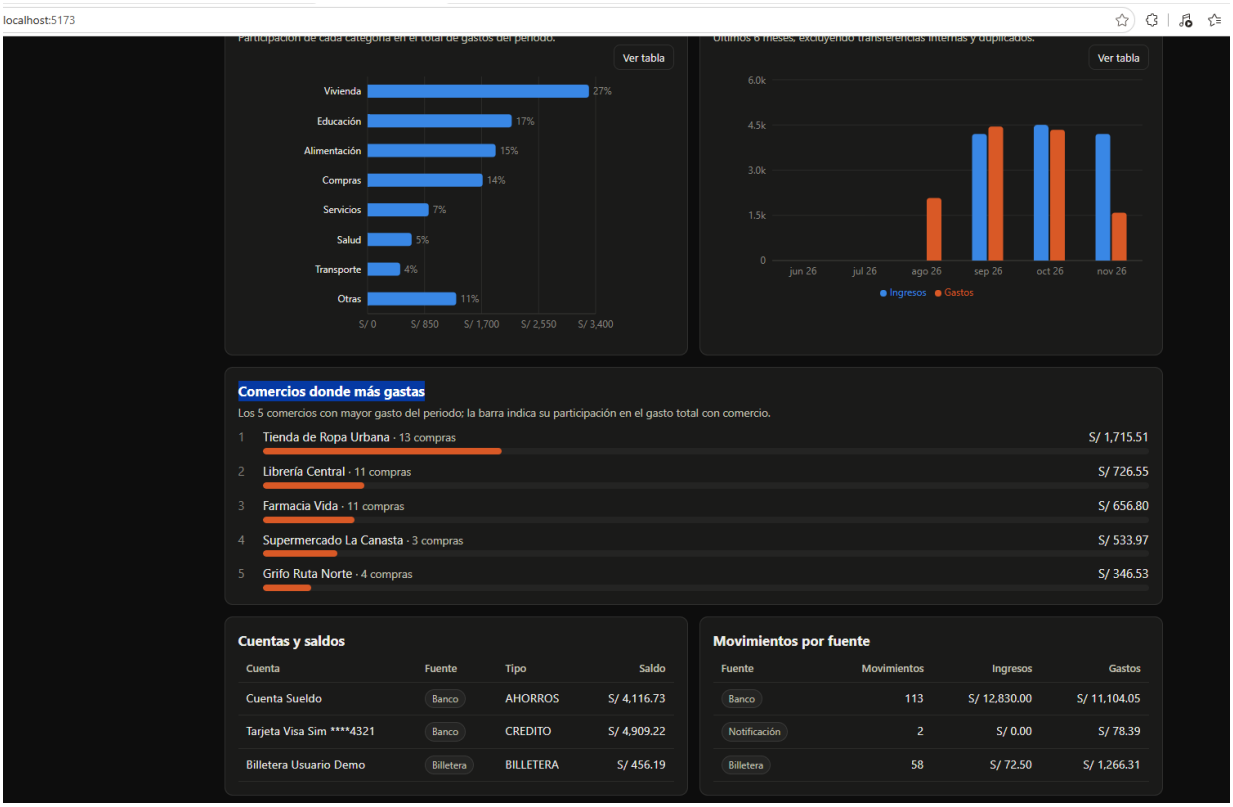



**Figura 35.** *Panel en tema oscuro con la tarjeta: los 5 comercios, número de compras, monto y barra proporcional.*

## 4.5 PR #3 revisado y fusionado

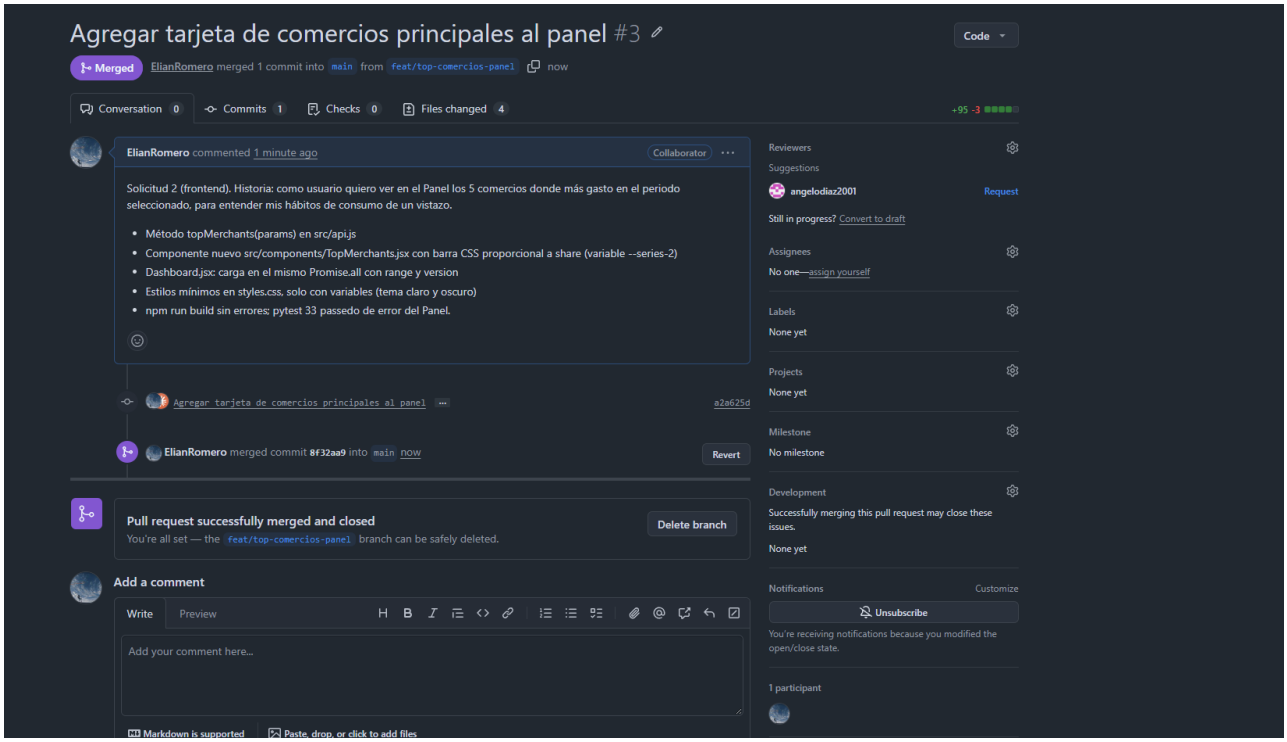



**Figura 36.** *PR #3 fusionado en main.*

*Estado final de main (git log)*

```
8f32aa9 Merge pull request #3 from angelodiaz2001/feat/top-comercios-panel
a2a625d Agregar tarjeta de comercios principales al panel
07c5bf1 Merge pull request #2 from angelodiaz2001/feat/top-comercios-api
6ffadc0 Agregar endpoint de comercios principales
4f1cd35 Merge pull request #1 from angelodiaz2001/chore/memoria-y-skills
240af54 Agregar memoria del proyecto y skills para Claude Code
```

**Resultado:** todo el trabajo llegó a main por 3 pull requests (#1, #2, #3), con commits pequeños en español, dos planes revisados antes de implementar (uno de ellos corregido por el grupo) y las pruebas pasando de 24 a 33.